Conhecendo o Kubernetes

Containers, orquestração e a arquitetura do K8S



Ada · DE-OP-011 Orquestração · Kubernetes | Aula 01

O que vamos ver hoje

- Por que os containers existem: o problema que eles resolvem
- O que é o Kubernetes e qual dor ele ataca
  - Comparação VM × Container × Kubernetes
- Arquitetura do cluster: Control Plane e Worker Nodes
- Os componentes essenciais e a analogia do navio de cargas
- Curiosidades e origem do projeto

VISÃO GERAL DO CURSO

O que vamos construir juntos?

8 aulas · do Minikube local ao Cluster Multi-Master em alta disponibilidade

Aula 1
Conhecendo o Kubernetes — arquitetura e fundamentos

Aula 2
Mão na Massa — Minikube e kubectl no seu computador

Aula 3
Pods, Namespaces e Controladores — objetos essenciais

Aula 4
Primeira Aplicação no K8S — deploy completo do zero

Aula 5
Estratégias de Deploy e Limitação de Recursos

Aula 6
Services, Ingress e Volumes — redes e persistência

Aula 7
Componentes e Troubleshooting — diagnóstico e RBAC

Aula 8
Cluster Multi-Master — Alta Disponibilidade com etcd HA

A dor antes dos containers

- Máquinas inteiras provisionadas para uma única aplicação — desperdício de recursos
- Ou várias aplicações na mesma máquina, sem limite de consumo por aplicação
  - Risco de conflito entre aplicações (dependências, versões de libs)
  - Alto custo de gerenciamento e mão de obra
- Ambientes difíceis de replicar: "na minha máquina funciona"

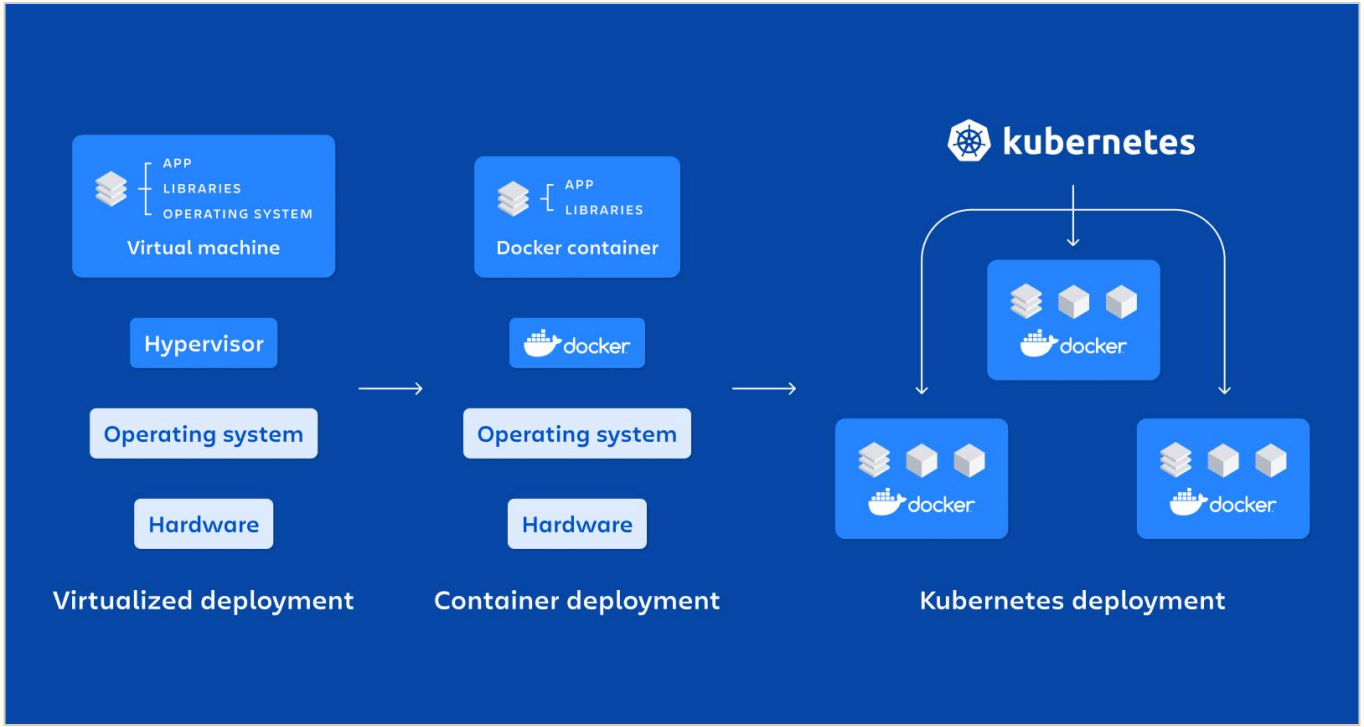
O que é um container

- Agrupamento isolado de recursos dentro de uma máquina
- Empacota a aplicação com TODAS as suas dependências (libs, runtime, config)
- Permite sistema de arquivos e ambiente distintos do host
  - Define limites de CPU, memória e disco por container
- Replicação rápida e segura do ambiente
- Várias aplicações na mesma máquina, sem conflito entre elas

VM × Container: a diferença fundamental

- VM: virtualiza o hardware — cada VM tem seu próprio sistema operacional completo
  - Pesada, sobe em minutos, consome muitos GB
- Container: virtualiza o SO — compartilha o kernel do host
  - Leve, sobe em segundos, consome MB
- Não são inimigos: é comum rodar containers DENTRO de VMs

VM × Container: a diferença fundamental



Fonte da imagem: https://www.atlassian.com/br/microservices/microservices-architecture/kubernetes-vs-docker

2. O que é o Kubernetes

Orquestração de containers em escala

Microsserviços trouxeram um novo problema

- Containers popularizaram a arquitetura de microsserviços
- Novo desafio: como gerenciar dezenas/centenas de containers automaticamente?
  - Como escalar quando a demanda sobe?
  - Como garantir que, se um container morre, outro sobe no lugar?
  - Como distribuir a carga entre eles?
- É aqui que entra a ORQUESTRAÇÃO de containers

Kubernetes (K8S): o que ele entrega

- Ferramenta open source para orquestração de containers
- Balanceamento de carga entre containers
- Garante um número desejado de réplicas sempre disponível (self-healing)
- Limites de recursos + estratégias de escalabilidade automática
- Estratégias de deploy para reduzir indisponibilidade
- Padrão de fato da indústria — mantido pela **CNCF** (Cloud Native Computing Foundation)
    - Organização sem fins lucrativos que faz parte da Linux Foudantion
    - A fundação apoia a comunidade de código aberto, ajudando com infraestrutura, marketing e orientação técnica para ferramentas que formam a espinha dorsal da computação moderna.

3. Arquitetura do Cluster

Control Plane, Worker Nodes e componentes

A analogia do transporte marítimo

- Cluster = um porto com vários navios
- Worker Nodes = os navios de carga (carregam os containers das aplicações)
- Master Nodes / Control Plane = guindastes e operadores (administram tudo)
  - Decidem quantos e quais containers vão em cada navio, e as rotas
- Kubernetes = "timoneiro" em grego (κυβερνήτης)



Componentes do Control Plane

- **kube-apiserver (A Porta de Entrada)**
    - **O que faz:** É o ponto central de comunicação do Kubernetes, tudo o que você pede para o cluster fazer passa obrigatoriamente por ele.
    - **Analogia:** É o **escritório central do porto**, se você quer colocar um novo contêiner no navio, você não vai direto ao navio; você entrega a documentação no escritório, e ele valida e autoriza o pedido.

Componentes do Control Plane

- **etcd (O Banco de Dados)**
  - **O que faz:** É um banco de dados extremamente confiável do tipo chave-valor, ele armazena absolutamente **todo o estado e as configurações** do seu cluster.
  - **Importância:** Se você perder o etcd e não tiver backup, você perde o cluster inteiro.
  - **Na analogia:** É o **livro de registro oficial do porto**, ele guarda a informação exata de quais navios estão ancorados, quais contêineres estão em cada navio e onde cada um deve ir.

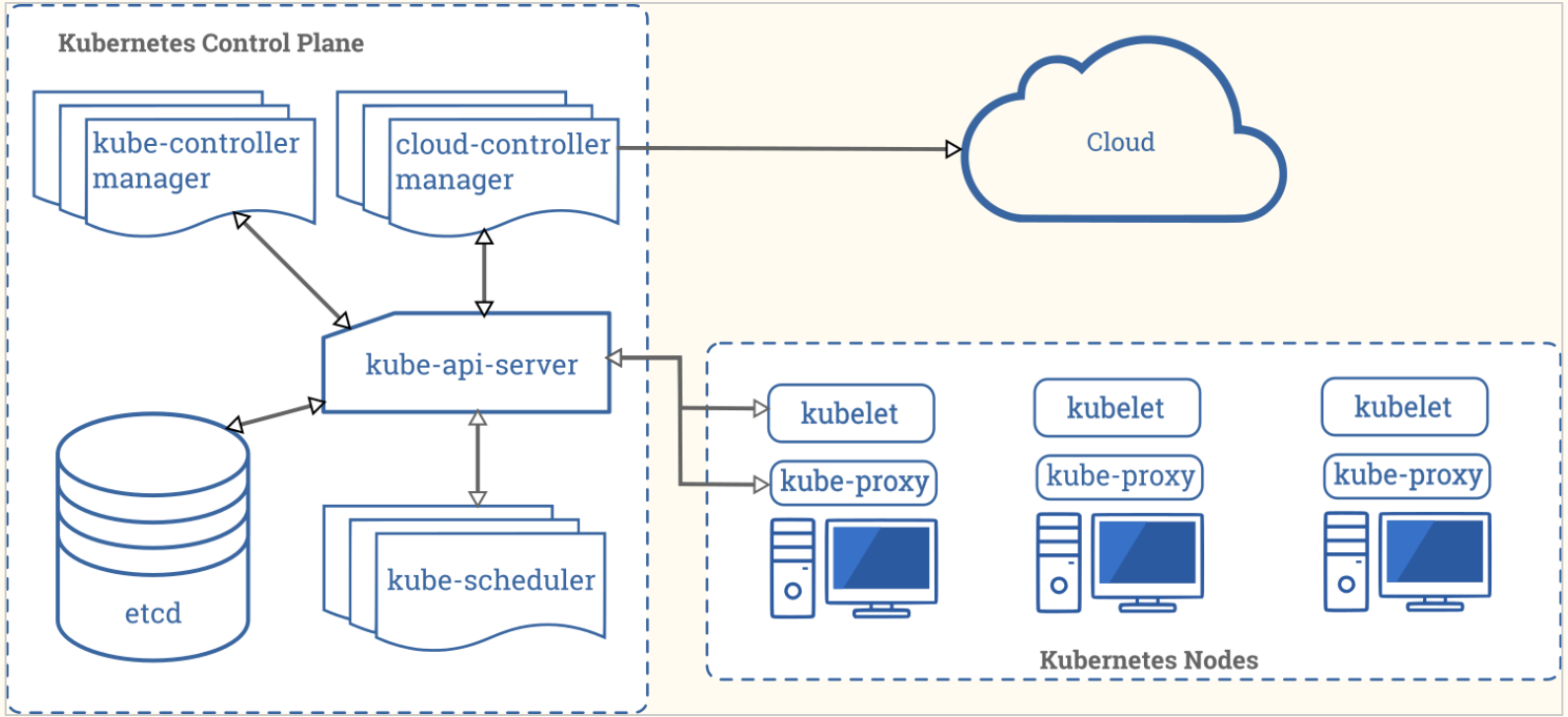
Componentes do Control Plane

- **kube-scheduler (O Tomador de Decisões)**
  - **O que faz:** Ele olha para os novos Pods (contêineres) que precisam ser criados e decide em qual Worker Node (navio) eles vão rodar, ele faz isso analisando os recursos disponíveis (como memória e CPU) e políticas específicas.
  - **Na analogia:** É o **planejador de carga**, ele analisa o tamanho e o peso do contêiner e decide em qual navio há espaço e estrutura adequados para transportá-lo.

Componentes do Control Plane

- **kube-controller-manager (O Fiscal do Estado Desejado)**
  - **O que faz:** É um conjunto de pequenos controladores que ficam rodando em loops contínuos, a única função deles é garantir que o estado atual do cluster seja exatamente igual ao estado desejado (definido por você).
  - **Como funciona:** Se você definiu que quer 3 cópias de um aplicativo rodando e uma delas falhar, o controller percebe isso imediatamente e ordena a criação de uma nova.
  - **Na analogia:** É o gerente de qualidade e segurança do porto, ele fica rondando o porto com uma prancheta garantindo que as regras sejam cumpridas e que nenhum contêiner caia no mar sem ser substituído.

Componentes do Control Plane



Fonte da imagem: http://kubernetes.io

Componentes de cada Node

- kubelet — agente em cada node; garante que os containers do Pod estão rodando
  - Não gerencia containers que não foram criados pelo Kubernetes
- kube-proxy — mantém as regras de rede; permite comunicação com os Pods
- Container runtime — o software que de fato executa os containers
  - Docker, containerd, CRI-O...

Pods: a menor unidade

- Pod = unidade básica do Kubernetes
- Encapsula um ou mais containers que compartilham rede e armazenamento
- Fazer deploy de uma aplicação = criar um ou mais Pods
- Aprofundaremos Pods, Namespaces e Controladores na Aula 3

4. Origem e curiosidades

De onde veio o Kubernetes

Curiosidades

- Lançado pela Google em 21 de julho de 2015
- Hoje mantido pela CNCF (Cloud Native Computing Foundation)
- Kubernetes = "timoneiro/piloto" em grego
- K8S: 8 letras entre o K e o S (K-ubernete-S)
- Herdeiro do Borg, sistema interno de orquestração da Google

Recapitulando a Aula 01

✓ Containers isolam aplicações e suas dependências, de forma leve

✓ Kubernetes orquestra containers em escala, mantendo o estado desejado

✓ Cluster = Control Plane (comando) + Worker Nodes (execução)

✓ Componentes-chave: apiserver, etcd, scheduler, controller-manager, kubelet, kube-proxy

✓ Próxima aula: instalar Minikube e kubectl — mão na massa!

